nginx的进程模型

nginx进程主要分为两种：**master和worker**

- **master**

  master是nginx的主进程，负责**读取和验证配置文件、管理worker进程**，只有一个

- **worker**

  工作进程，负责处理实际请求，可以有多个

- **查看当前进程**

  ```
  ps -ef|grep nginx
  ```

  输出后可以看见如下返回：

  ```
  root       13435       1  0 20:47 ?        00:00:00 nginx: master process nginx
  www-data   13436   13435  0 20:47 ?        00:00:00 nginx: worker process
  www-data   13437   13435  0 20:47 ?        00:00:00 nginx: worker process
  www-data   13438   13435  0 20:47 ?        00:00:00 nginx: worker process
  www-data   13439   13435  0 20:47 ?        00:00:00 nginx: worker process
  ellery     13672    4401  0 20:47 pts/1    00:00:00 grep --color=auto nginx
  ```

  其中第二列的五位数字是PID，这是Linux系统每一个进程的唯一标识

- 端口

  nginx默认监听80端口

- **退出相关**

  ```
  nginx -s quit   #正常停止
  nginx -s stop   #立即强制停止
  nginx -s reload #重载配置文件
  nginx -s reopen #重新打开日志文件
  ```

反向代理和负载均衡

1. **反向代理**

   反向代理是nginx核心功能之一，意思是代理服务端（正向代理表示代理客户端）

   比如一个网站有多个服务器，但显然用户只会通过一个域名来访问网站，这时候，用户通过向服务端的代理发送请求，让它给各个服务器转发请求，这就是反向代理，**它的核心命令是**`proxy_pass 目标地址`

2. **负载均衡**

   在有多台后端服务器时，我们需要反向代理转发请求，但是如何分配请求呢？这就是负载均衡的意义。

   负载均衡可以实现**分摊压力、提升可用性、故障自动避让**等等功能

   负载均衡的核心命令是upstream，格式例子如下（它一般写在server块的外面）：

```
upstream backend_servers {
    server 127.0.0.1:8080 weight=3;
    server 127.0.0.1:8081;
    server 127.0.0.1:8082;
}
```

upstream用来**定义一组后端服务器**，如果只有proxy_pass反向代理，nginx实际上只能转发请求到一个后端服务器

而有了upstream之后，nginx就可以在这些服务器之间去分配请求，负载均衡

upstream里面的server不仅仅可以写地址，还可以带一些参数，比如第一行的weight=3，这表示权重，第一行服务器的权重是其他的三倍

nginx的命令

1. **快速定位配置文件位置&检查配置文件**

```
nginx -t
```

这个命令不仅可以查看配置文件位置，还可以看后面行查看配置文件是否修改成功、是否修改正确

如果有错误，他会告诉你

不要忘记`nginx -s reload`重载一下配置文件

2. **启动nginx服务**

```
sudo systemctl start nginx
```

3. **(非nginx命令)检查端口是否有被监听**

```
sudo ss -tulpn | grep 8000
```

nginx配置文件的结构

nginx的配置文件主要分为三个大块

1. **全局块**

```
user www-data;
worker_processes auto;
pid /run/nginx.pid;
error_log /var/log/nginx/error.log;
include /etc/nginx/modules-enabled/*.conf;
```

负责规定一些全局配置，比如worker_processes这一栏就是规定worker进程数量的

2. **events块**

```
events {
    worker_connections 768;
    # multi_accept on;
}
```

服务器、客户端之间网络连接的配置

worker_connections 768表示每个worker进程可以接受768个网络连接

3. **http块**

主要修改内容在这里，它是用来**配置 HTTP/HTTPS 网站服务的大模块**

网站相关的规则、日志虚拟主机`server{}`全部写在http块内部，对所有HTTP站点生效

我还没有进行一些配置，所以http块看起来不完善

其中，里面可以包含：

1. **server块**

```
server {
    listen 80;

    location / {
        root /usr/share/nginx/html;
    }

    location /api/ {
        proxy_pass http://backend:8080;
    }
}
```

server块表示一个虚拟主机，可以说，一个`server{}`表示一个独立网站

其中，**listen表示监听的端口**

location

location是server的子块，用来**匹配访问的URI路径**，针对不同url路径设置规则

这里`location /`表示匹配所有请求，`location /api/`表示只匹配以`/api/`开头的请求

`root /usr/share/nginx/html;`表示网站文件的根目录

`proxy_pass http://backend:8080;`这里是反向代理做法

proxy_pass作用：把请求转发到后端服务

此外，还有`index`这个指令，可以**定义默认主页文件**
当用户访问路径是一个**目录**而非具体文件，nginx会自动找index指定文件返回

2. **include**

   有很多include在http里面，比如

   ```
   include /etc/nginx/mime.types;
   ```

   它们的作用是**把外部其他文件的配置内容，直接导入、粘贴到当前这个位置**

   代码块里面那一行代表把文件后缀对照表下载下来

   作用是**让浏览器拿到相应后缀的文件之后，知道如何处理它**，比如html作为文本文件处理

3. **其他配置**

   ```
   sendfile on;
   ```

开启内核零拷贝（具体我也不清楚是啥ww），反正读取静态文件（html、图片等等）更快，　　适合静态网站

```
access_log / error_log
```

access_log：记录每一次正常访问（访问 IP、时间、url、状态码）

error_log：404、403、配置错误等异常日志